

UnifiedXRMotion: A Validity-Aware Runtime Contract for Heterogeneous XR Avatar Motion

Anonymous Authors

Paper under double-blind review

Abstract. XR avatar-motion pipelines often combine sources that differ in body coverage, joint layout, coordinate convention, and component availability. A headset, controller pair, articulated hand tracker, vision model, replay stream, inferred-motion module, and retargeting node may all produce motion for the same avatar pipeline, but downstream consumers should not need source-specific logic for every case. We present UnifiedXRMotion, a Unity prototype organized around a validity-aware runtime contract for heterogeneous XR avatar motion. The contract uses a canonical skeletal schema, reusable sub-skeleton views, and per-joint, per-component availability bits. Observed, inferred, replayed, synthetic, and retargeted components flow through the same skeletal representation; provenance remains implicit in the producing node rather than encoded in the data record. SDK and vision adapters map native observations into the contract, while downstream retargeting, recording, and optional transport components consume shared views and gate reads on explicit availability. Evaluation combines implementation evidence, adapter/producer coverage, reusable-consumer evidence, and a within-subjects setup-burden study. The results support a scoped systems contribution: a concrete contract and boundary for composing selected heterogeneous XR avatar-motion sources without requiring downstream consumers to depend on SDK-specific records.

Keywords: Extended reality · Avatar motion · Tracking middleware · Motion-data contract · Runtime systems · Authoring

1 Introduction

XR avatars depend on motion pipelines that span tracking acquisition, skeletal representation, motion processing, retargeting, recording, optional transport, and rendering. In practice these stages are often tied to one SDK or one avatar asset. A hand tracker may expose a vendor-specific joint order; a body tracker may report only a subset of joints; a vision model may provide positions but not reliable rotations; a replay stream may preserve previously recorded availability; and an inference or retargeting node may publish components that were not directly observed. This heterogeneity is not merely inconvenient engineering. It makes it hard to replace trackers, compare pipelines, record and replay motion, or reuse downstream avatar consumers without rewriting glue code.

Existing XR APIs and tracking middleware address important parts of this problem. OpenXR standardizes access to XR devices and extensions such as hand tracking [6, 7]. Earlier VR middleware, including VRPN [13], OpenTracker [11], and Ubiquitous Tracking [9], showed that device abstraction and distributed tracking are valuable architectural tools. UnifiedXRMotion builds on that lesson at a narrower layer: avatar-motion pipelines that must carry incomplete skeletal components through adapters, filters, inferred-motion producers, retargeters, record/replay utilities, and optional network wrappers.

The central design choice is to separate *slot identity* from *component availability*. A schema defines addressable motion slots and region views; a producer publishes the slots and components it has available; each pose component is guarded by a validity bit; and downstream consumers read only the components whose bits are set. This contract does not assert that a component is ground-truth correct, directly measured, or high-confidence. It only states that the producing node has made that component available for downstream consumption under its own semantics. In the current prototype, the contract is instantiated with a 77-slot full-body schema covering torso, limbs, head, and articulated hands. This layout is one implementation instance rather than a general skeleton standard.

This paper makes three contributions:

- **A validity-aware skeletal motion contract** that combines stable slot identities, a canonical 77-slot prototype schema, sub-skeleton views, per-slot pose records, and per-component availability flags.
- **A heterogeneous producer and adapter boundary** that maps selected XR, vision, replay, synthetic, inferred-motion, and retargeted producers into the shared representation while localizing source-specific joint ordering, coordinate conversion, body coverage, and missing-component semantics.
- **Validity-gated downstream consumption and setup evidence**, demonstrated by retargeting, recording, and optional transport consumers that use shared contract views, plus a within-subjects Unity setup-burden study for the tested authoring task.

2 Related Work

Tracking middleware and XR APIs. VRPN provides a device-independent and network-transparent interface for VR peripherals [13]. OpenTracker introduced a modular software architecture for tracking configurations [11], and Ubiquitous Tracking modeled distributed spatial relationships as a graph of tracking transformations [9]. OpenXR standardizes XR device access and extensions such as hand tracking [6, 7]; platform SDKs add body, hand, controller, camera, or device-specific streams [8, 15, 17, 4]. These systems abstract devices, transforms, or API access. UnifiedXRMotion instead targets a skeletal avatar-motion record in which individual pose components may be unavailable and consumers explicitly gate reads using component availability.

Avatar motion, retargeting, and inference. Sparse avatar-motion reconstruction and motion priors have advanced rapidly [5, 3, 18, 19, 12, 14]. Avatar representation, fidelity, and retargeting affect embodiment and agency [20, 2, 16], and AvatarGo studies plug-and-play self-avatar setup [10]. UnifiedXRMotion does not introduce a new tracker, IK solver, retargeting algorithm, or codebook method. Instead, textbook IK [1], codebook-style inference inspired by prior work [12], and retargeting components are treated as representative nodes that consume and republish the same motion contract.

3 Validity-Aware Motion Contract

UnifiedXRMotion treats skeletal motion as a runtime contract between motion producers and consumers. The contract consists of a schema, region views, per-slot pose records, and per-component availability bits. A slot is a stable address in the motion representation, such as a head, wrist, fingertip, tracker, prop, facial control, or application-specific joint. Each slot may carry position, rotation, velocity, and angular velocity components; each component has an explicit bit indicating whether it is available in the current record.

A contract instance separates static schema from runtime state. The static schema is

$$\mathcal{S} = (S, P, R, K), \quad (1)$$

where $S = \{s_0, \dots, s_{n-1}\}$ is a finite set of motion slots, P is an optional parent relation, R is a set of named region views with each $r_j \subseteq S$, and K is the set of pose-component kinds. A runtime sample stores

$$M = \{(m_i, V_i)\}_{i=0}^{n-1}, \quad m_i = (p_i, q_i, v_i, \omega_i), \quad V_i = (b_i^p, b_i^q, b_i^v, b_i^\omega), \quad (2)$$

with $b_i^k \in \{0, 1\}$. A consumer may read component m_i^k only when $b_i^k = 1$. The prototype implements this rule through `PhysicBone`, which stores pose fields and a `PoseFlag` bitmask, and through `KinematicBuffer<T>` and `BodyType` views over full-body, upper-body, lower-body, two-arm, two-hand, and hand regions.

This distinction is essential for inferred and retargeted motion. A lower-body inference node may publish a rotation as available, and a retargeting node may publish an avatar-specific component as available, but the contract does not encode whether that component was observed, inferred, reconstructed, replayed, or retargeted. Provenance is implicit in the producing node. Likewise, a vision adapter may threshold continuous confidence into a binary availability bit, but the continuous confidence value is not preserved in the current record. These are deliberate scope boundaries for the present prototype.

4 Heterogeneous Producers and Adapter Boundary

Producers publish motion into the contract. Device and SDK adapters are the boundary where native tracking outputs become contract records. Each adapter

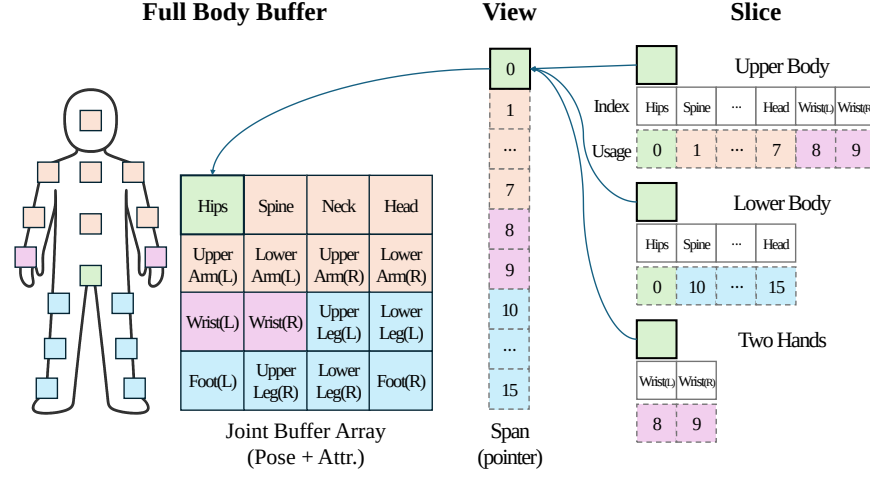


Fig. 1. Prototype buffer and sub-skeleton-view instantiation of the contract. A canonical full-body buffer is exposed through region views so that producers and consumers can operate on the subset they require.

initializes or reads a source, maps native joint ordering into schema slots, converts coordinate conventions where needed, marks component availability, and publishes pose records. The adapter code remains vendor-specific wrapper and conversion logic; the reusable artifact is the boundary after publication, where downstream nodes depend on slot identity and component availability rather than native SDK records.

The prototype exposes a minimal adapter interface through `ITrackingDeviceHandler`: initialization, a set of tracked schema slots, a tracking-information buffer, and a tracking update. Its producer portfolio includes OVR/Meta body and hand streams, Unity XR/OpenXR HMD, controller, and hand sources, XREAL/NRSDK and Vive hand sources, Mediapipe vision sources, Android-AR or scene-transform sources, replay streams, synthetic sources, and representative inferred or retargeted producers. Table 3 summarizes the evidence used in the medium-paper framing.

The adapter boundary localizes several classes of heterogeneity: vendor joint ordering, handedness and axis conversion, body-region coverage, missing velocity or angular-velocity channels, and SDK tracking-state or visibility conversion. The boundary is not leak-free. For example, vision visibility is reduced to a binary bit, and some device adapters contain source-specific offsets or calibration assumptions. These limitations are why the claim is limited to selected producer families and explicit component availability, not to broad cross-device coverage or automatic calibration.

Table 1. Contract concepts and evidence scope. Validity denotes component availability, not correctness, confidence, or provenance.

Concept	Prototype evidence	Scope boundary
Schema and slots	77-slot full-body prototype schema with parent maps and sub-skeleton views.	One schema instantiation; arbitrary rigs require schema agreement and retargeting.
Pose record	Per-slot PhysicBone fields for position, rotation, velocity, scalar angular velocity, and local transforms.	The record does not carry timestamp, continuous confidence, or source provenance.
Availability	PoseFlag marks position, rotation, velocity, and angular-velocity availability per slot.	Availability does not imply ground-truth correctness or direct sensor observation.
Region views	KinematicBuffer<T> and BodyType expose full-body, upper/lower-body, arms, hands, and hand views.	Views are implementation-defined rather than discovered from arbitrary avatars.
Serialization	Optional transport writes the flag first and can omit unavailable components.	Used as semantic preservation; this paper does not frame the contribution as compression.

Table 2. Operational invariants used to keep the contract explicit. These are implementation-level obligations for the prototype, not claims of correctness, provenance, or uncertainty modeling.

Invariant	Required behavior	Scope boundary
Producer availability	A producer sets component flags for the values it publishes into a slot.	The bit denotes availability under producer semantics, not measured accuracy.
Consumer gating	A consumer checks the relevant flag before reading or assigning a pose component.	The paper demonstrates representative consumers rather than proving all package modules are SDK-agnostic.
View consistency	Region views preserve slot identity when exposing full-body, hand, upper-body, or lower-body subsets.	The slot map is a prototype schema, not a universal avatar ontology.
Availability preservation	Recording and optional transport preserve flags together with published components.	Transport is semantic support, not a latency, compression, or equivalence contribution.

5 Validity-Aware Consumers

The strongest consumer evidence is the retargeting path. **RetargetSystem** reads **KinematicBufferView<PhysicBone>** instances and checks **PoseFlag** before assigning position or rotation to avatar transforms. If a component is unavailable, the assignment is skipped rather than silently using a default or stale source value. The same consumer dispatches across **BodyType** views such as full body, upper/lower body, two arms, and two hands, so the consuming code is organized around contract views rather than around a producing SDK.

This consumer evidence is deliberately narrower than claiming that every module is SDK-agnostic. Some concrete retargeters or adapters are vendor-specific by design. The medium-paper claim is that the common contract gives downstream consumers a shared availability-gated representation, and that representative retargeting, recording, inference, replay, and transport nodes operate on that representation without depending on each native SDK record format.

Table 3. Producer and adapter evidence. The table reports selected implementation evidence, not broad device coverage.

Producer family	Source examples	Contract mapping	Important limitation
Observed XR devices	Meta/OVR body and hand, Unity XR/OpenXR HMD/-controllers/hands, Vive hands, XREAL hands.	Native joint IDs and device poses are mapped into full-body or region slots with per-component availability.	Adapter logic is SDK-specific; coverage is limited to selected source families.
Vision sources	Mediapipe upper-body and hand landmarks.	Landmark positions are mapped into hand or upper-body slots; visibility is thresholded into binary availability.	Continuous confidence is quantized away; coordinate offsets remain adapter assumptions.
Replay and synthetic sources	Recorded <code>PhysicBone</code> arrays, transform sources, dummy or mouse-driven inputs.	Publish the same slot records and flags as live sources.	Fidelity depends on the recorded stream or synthetic setup.
Inferred producers	Codebook-style lower/upper-body regressors and filters.	Publish inferred components back into <code>KinematicBuffer</code> < <code>PhysicBone</code> with availability flags.	Inference algorithm and model quality are not contributions; provenance is not encoded in the record.
Retargeted producers	IK and retargeting nodes that fill avatar-motion regions.	Consume views and may republish retargeted components into the same representation.	IK and retargeting methods are representative nodes, not algorithmic novelty.

Table 5 makes the reuse claim explicit by linking producer families to the contract view consumed downstream. The table is not a device-coverage benchmark; it shows that different production paths can publish into the same record/view abstraction, after which consumers branch on availability and region view rather than on the native SDK record.

6 Prototype Implementation

UnifiedXRMotion is implemented as a Unity package. Motion slots are stored in preallocated buffers; region views expose subsets without requiring a separate representation for each body part. Root producers are advanced by a frame-driven trigger list, and motion updates propagate through sender/receiver events. This runtime organization is a Unity-oriented composition mechanism rather than a general graph scheduler or graph optimizer.

The package includes representative nodes for filtering, IK, retargeting, codebook-style inference, record/replay, and optional FishNet-based transport. Textbook IK and prior codebook-style control provide algorithmic background [1, 12]; the contribution here is that these nodes can consume or publish the same validity-aware representation. Optional transport preserves flags and available components, but transport is treated as implementation support rather than the center of the paper.

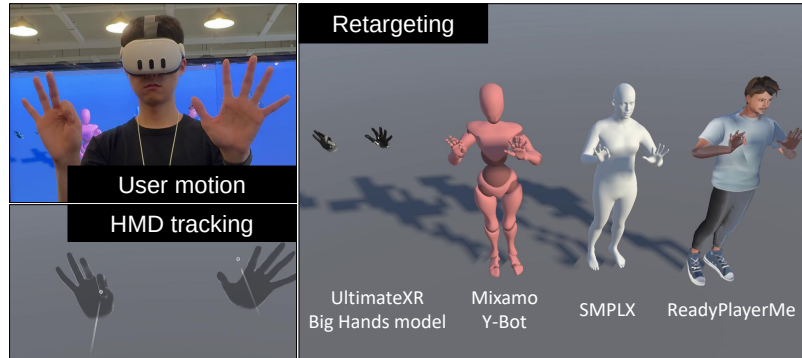


Fig. 2. Representative retargeting consumers reading contract views and writing to avatar rigs. Retargeting is used as evidence of validity-gated consumption, not as an algorithmic contribution.

Table 4. Downstream consumers and evidence scope.

Consumer	Contract use	Supported claim	Unsupported claim
Retargeting system	Reads region views and gates transform assignment on PoseFlag .	Same consumer structure can handle multiple sub-skeleton views.	New retargeting algorithm or avatar-quality guarantee.
Recording/replay	Stores and republishes PhysicBone arrays with flags.	Replay can act as another producer in the same representation.	Ground-truth motion or accuracy benchmark.
Network serialization	Writes PoseFlag before optional channels and preserves availability semantics.	Optional transport can carry partial records.	Compression result, runtime-speed result, or transparent equivalence.
Filters/inference nodes	Read upstream views and republish processed or inferred components.	Observed and inferred components flow through one representation.	Encoded-provenance or uncertainty processing.

7 Evaluation and Case Study Evidence

The evaluation is scoped to the medium-paper claim. It asks whether the contract is instantiated in a concrete runtime, whether selected heterogeneous producers can be localized at adapters while consumers reuse contract views, and whether the authoring workflow reduces procedural setup burden in one controlled Unity task. The evaluation does not measure tracking accuracy, retargeting quality, general developer productivity, broad device coverage, or display-response timing.

Contract, producers, and consumers. Tables 1, 2, 3, 4, and 5 summarize the implementation evidence. The contract evidence is the record/view/flag mechanism; the producer evidence is the selected adapter and producer portfolio; and the consumer evidence is validity-gated consumption through retargeting, recording, inference/replay, and optional transport. This evidence supports a

Table 5. Producer-to-consumer reuse through shared contract views. Rows show representative paths that support the medium-paper claim; they do not imply universal source coverage or encoded provenance.

Producer path	Contract view	Downstream consumer	Claim supported
Meta/OVR body tracking	Full-body or upper/lower-body view	Retargeting system	Vendor body streams can be consumed through schema views after adapter mapping.
UnityXR/OpenXR hand sources	Two-hands or hand view	Hand retargeting and recording	Hand-only streams use the same slot/view structure as other producers.
Mediapipe vision landmarks	Position-available upper-body or hand view	Filters, recorder, optional transport	Position-dominant vision sources can publish partial component availability without requiring special downstream records.
Codebook-style inference	Lower-body or full-body view	Retargeting and recorder	Inferred components flow through the same representation, although the record does not encode inferred provenance.
Replay or synthetic source	Recorded or generated PhysicBone buffer	Same representative consumers	Replay and test inputs can act as producers without changing the consumer interface.
Retargeting or IK node	Region view republished as a buffer	Later recording or optional transport	Retargeted components can be republished as available components without claiming a new retargeting algorithm.

concrete runtime contract and boundary, not a claim that the system solves all avatar-motion interoperability problems.

Setup-burden study. We conducted a within-subjects Unity setup task comparing a UnifiedXRMotion workflow against a vendor SDK workflow. Participants configured the same scene-level goal in both conditions: making custom OpenXR hand prefabs follow simulated hand motion and making the same Y Bot avatar follow simulated full-body motion. Both conditions used the Meta XR Simulator for verification. The study used Unity 6000.0.33f1 and Meta Core, Movement SDK, and XR Simulator version 78.0.0.

To avoid confounding pipeline assembly with mesh-specific hand-axis calibration, custom hand retargeting and axis alignment were completed before each session. Participants were instructed to prioritize correctness and completion rather than speed. We therefore interpret completion time as procedural setup burden for the tested task, not as a general developer-speed estimate. The procedure also excludes adapter authoring and full integration of a previously unseen hand mesh.

Table 6 shows lower completion time and lower workload for the UnifiedXR-Motion condition, with higher SUS scores. The completion-time result had an order-sensitive pattern across counterbalanced orders (mean difference -307.6 s for AB and -521.4 s for BA; Welch test $t(16.94) = 3.32$, $p = .0040$). Sequence sensitivity was not observed for NASA-TLX ($t(16.10) = -0.31$, $p = .763$) or

Table 6. Within-subjects setup-burden results ($N = 19$). Values are mean $\pm$ standard deviation. NASA-TLX is the 0–100 workload score after reversing the performance subscale so that higher values consistently indicate higher workload. Completion time is interpreted as procedural setup burden for the tested Unity task.

Statistic	Completion time (s)	NASA-TLX	SUS
UnifiedXRMotion ($M \pm SD$)	564.03 $\pm$ 176.58	30.53 $\pm$ 17.55	73.42 $\pm$ 17.08
Vendor SDK ($M \pm SD$)	972.89 $\pm$ 243.36	49.74 $\pm$ 18.29	34.47 $\pm$ 13.91
95% CI (UXM – SDK)	[−493.25, −324.48]	[−24.87, −13.55]	[28.69, 49.21]
$t(df)$	−10.18 (18)	−7.13 (18)	7.98 (18)
p	< .0001	< .0001	< .0001
Hedges’ g_{av}	−1.84	−1.03	+2.39

Statistics are computed on paired differences from the supplied raw CSVs. The study supports setup burden in this prepared task; it does not establish general developer productivity or long-term maintainability.

SUS ($t(16.99) = 1.53$, $p = .145$). The setup-burden claim is therefore useful but narrow: it supports the tested procedural configuration task, not broad claims about developer productivity.

8 Discussion and Limitations

UnifiedXRMotion is evaluated as an implemented contract boundary, not as a standardized interchange format or an end-to-end performance system. The core record does not encode provenance, continuous confidence, schema version, or full timestamp propagation. Provenance is implicit in the producing node, so a downstream consumer cannot tell from the data record alone whether a component was directly observed, inferred, replayed, reconstructed, or retargeted. Vision confidence is collapsed into binary availability in the current adapters, losing information that future versions should preserve.

The adapter portfolio is selected rather than exhaustive. Each new tracker still requires a schema mapping, coordinate conversion, and validation, and some mappings contain source-specific assumptions such as offsets or calibration conventions. The 77-slot schema is one prototype instantiation, not a general avatar skeleton. Arbitrary avatar rigs require schema agreement and retargeting between the runtime representation and the avatar-specific skeleton. The included IK, retargeting, filtering, and codebook-style modules are representative nodes, not algorithmic contributions.

The setup study also has limited scope. Hand-axis alignment and mesh-specific retargeting were pre-completed, participants worked on a prepared Unity task, and the study did not measure adapter-authoring effort, long-term maintenance, or quality of produced motion. Optional local/remote traces and transport utilities characterize application-level runtime behavior only; they should not be interpreted as display-response timing, remote-execution performance benefit, or user-perceived responsiveness.

9 Conclusion

UnifiedXRMotion reframes heterogeneous XR avatar-motion integration as a validity-aware runtime contract problem. Producers publish observed, inferred, replayed, synthetic, or retargeted components into a shared skeletal representation, and consumers read only explicitly available components through schema views. The Unity prototype demonstrates the contract with selected producer families, adapter mappings, reusable retargeting/recording/transport consumers, and a controlled setup-burden study. The evidence supports a scoped EuroXR medium-paper contribution: a concrete contract and adapter boundary for composing heterogeneous XR avatar-motion pipelines without making downstream consumers depend on SDK-specific motion records.

References

1. Aristidou, A., Lasenby, J.: FABRIK: A fast, iterative solver for the inverse kinematics problem. *Graphical Models* **73**(5), 243–260 (2011)
2. Cheymol, A., Fribourg, R., Lécuyer, A., Normand, J.M., Argelaguet, F.: Beyond my real body: Characterization, impacts, applications and perspectives of “dissimilar” avatars in virtual reality. *IEEE Transactions on Visualization and Computer Graphics* **29**(11), 4426–4437 (2023)
3. Dai, P., Zhang, Y., Liu, T., Fan, Z., Du, T., Su, Z., Zheng, X., Li, Z.: HMD-Poser: On-device real-time human motion tracking from scalable sparse observations. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 874–884 (2024)
4. Google Developers: Introducing Android XR SDK developer preview. <https://developers.googleblog.com/en/introducing-android-xr-sdk-developer-preview/> (2024)
5. Jiang, J., Strelly, P., Qiu, H., Fender, A., Laich, L., Snape, P., Holz, C.: AvatarPoser: Articulated full-body pose tracking from sparse motion sensing. In: *Computer Vision – ECCV 2022. Lecture Notes in Computer Science*, vol. 13665, pp. 443–460. Springer (2022). https://doi.org/10.1007/978-3-031-20065-6_26, https://link.springer.com/chapter/10.1007/978-3-031-20065-6_26
6. Khronos Group: Khronos releases OpenXR 1.0 specification. <https://www.khronos.org/news/permalink/khronos-releases-openxr-1.0-specification> (2019), press release
7. Khronos Group: XR_EXT_hand_tracking. https://registry.khronos.org/OpenXR/specs/1.1/man/html/XR_EXT_hand_tracking.html (2025)
8. Meta Horizon: Openxr support for meta quest headsets. <https://developers.meta.com/horizon/documentation/native/android/mobile-intro/> (2025)
9. Newman, J., Wagner, M., Bauer, M., MacWilliams, A., Pintaric, T., Beyer, D., Pustka, D., Strasser, F., Schmalstieg, D., Klinker, G.: Ubiquitous tracking for augmented reality. In: *Proceedings of the 3rd IEEE and ACM International Symposium on Mixed and Augmented Reality*. pp. 192–201. IEEE Computer Society (2004)
10. Ponton, J.L., Monclús, E., Pelechano, N.: AvatarGo: Plug and play self-avatars for VR. In: *Eurographics 2022 – Short Papers*. pp. 77–80. The Eurographics Association (2022). <https://doi.org/10.2312/egs.20221037>

11. Reitmayr, G., Schmalstieg, D.: An open software architecture for virtual reality interaction. In: *Proceedings of the ACM Symposium on Virtual Reality Software and Technology*. ACM (2001)
12. Starke, S., Starke, P., He, N., Komura, T., Ye, Y.: Categorical codebook matching for embodied character controllers. *ACM Transactions on Graphics* **43**(4), 142:1–142:14 (2024). <https://doi.org/10.1145/3658209>
13. Taylor, R.M., Hudson, T.C., Seeger, A., Weber, H., Juliano, J., Helser, A.T.: VRPN: A device-independent, network-transparent VR peripheral system. In: *Proceedings of the ACM Symposium on Virtual Reality Software and Technology*. ACM (2001)
14. Tessler, C., Juravsky, J., Guo, Y., Jiang, Y., Coumans, E., Peng, X.B.: ProtoMotions: Physics-based character animation. <https://github.com/NVlabs/ProtoMotions/> (2024)
15. Vive: VIVE OpenXR All-in-One/PCVR. <https://developer.vive.com/resources/openxr/> (2024)
16. Weidner, F., Boettcher, G., Arevalo Arboleda, S., Diao, C., Sinani, L., Kunert, C., Gerhardt, C., Broll, W., Raake, A.: A systematic review on the visualization of avatars and agents in AR & VR displayed using head-mounted displays. *IEEE Transactions on Visualization and Computer Graphics* **29**(5), 2596–2606 (2023). <https://doi.org/10.1109/TVCG.2023.3247072>
17. XREAL: XREAL SDK 3.0.0 release notes. <https://docs.xreal.com/Release/%20Note/XREAL/%20SDK/%203.0.0> (2024)
18. Xu, V., Gao, C., Hoffmann, H., Ahuja, K.: MobilePoser: Real-time full-body pose estimation and 3D human translation from imus in mobile consumer devices. In: *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*. pp. 1–11 (2024)
19. Yang, D., Kang, J., Ma, L., Greer, J.D., Ye, Y., Lee, S.H.: DivaTrack: Diverse bodies and motions from acceleration-enhanced three-point trackers. *Computer Graphics Forum* **43**(2) (2024). <https://doi.org/10.1111/cgf.15057>
20. Yun, H., Ponton, J.L., Andujar, C., Pelechano, N.: Animation fidelity in self-avatars: Impact on user performance and sense of agency. *IEEE Transactions on Visualization and Computer Graphics* (2023)